

Relational Database Design

Join Dependencies

Uni Auckland Databases 51

Gerald Weber

Recap: Decomposition, Example 2

- ▶ Decomposition of a schema R into schemas $R_1, R_2, \dots, R_n$.
- ▶ Consider the relation **Course(course_id, hod_name, dept_name, credits)**.

▶ Tuples:

course_id	hod_name	dept_name	credits
C1	Pat	CS	4
C2	Pat	CS	3
C3	Kim	Math	3

- ▶ The relation can be reconstructed by joining:
 - ▶ **Course1(course_id, dept_name)**
 - ▶ **Course2(dept_name, hod_name)**
 - ▶ **Course3(course_id, credits)**

Model

- ▶ In the model assumed in Relational Database Design, a relation can be constrained by business considerations in a certain way:
- ▶ In the example before the fact that the HoD name should be the same whenever a department appears is such a constraint.
- ▶ We often say, only relations are *legal* where this pattern is maintained.
- ▶ This constraint usually cannot be enforced in the database:
- ▶ This is one of the reasons why such a table experiences anomalies with any kind of change:
- ▶ Code that changes the table is more complicated than necessary, error prone and inefficient.
- ▶ This situation is considered non-optimal and the decomposition is preferred.

Aim

见前 PPT

- ▶ If a relation schema R can be reconstructed just by natural join from a set of schemata $R_1, R_2, \dots, R_n$ with all of them subsets of R :
- ▶ then we prefer a database with relations based on $R_1, R_2, \dots, R_n$
- ▶ we have to identify what concerns we have to take into account in this process.

Update, Insert, Delete Anomalies 已读

► **Update Anomaly:**

- Occurs when data redundancy causes inconsistencies.
- Example: Changing the HoD name for the CS department requires updating multiple tuples. If one tuple is missed, the data becomes inconsistent.

► **Insert Anomaly:**

- Occurs when certain attributes cannot be inserted into the database without the presence of other attributes.
- Example: Adding a new department without any courses would be impossible if the course information is required.

► **Delete Anomaly:**

- Occurs when deletion of data representing certain facts necessitates deletion of data representing other facts.
- Example: Deleting the last course of a department also removes the department's HoD information.

Inferring decompositions from Data

推測

- ▶ We frequently infer heuristically such constraints from database instances and the patterns we can see.
- ▶ e.g. **Course(course_id, hod_name, dept_name, credits).**

▶ Tuples:

course_id	hod_name	dept_name	credits
C1	Pat	CS	4
C2	Pat	CS	3
C3	Kim	Math	3

推測

- ▶ We assume the repeated name Pat is not a coincidence.
- ▶ Once we made this observation, we assume only relations are legal where this pattern is maintained. This also means, attributes have meaning.
 - ▶ **Course1(course_id, dept_name)**
 - ▶ **Course2(dept_name, hod_name)**
 - ▶ **Course3(course_id, credits)**

Formalization

- ▶ Note $\text{Course}(\text{course_id}, \text{hod_name}, \text{dept_name}, \text{credits})$ both defines Course and states that it has the schema $\{\text{course_id}, \text{hod_name}, \text{dept_name}, \text{credits}\}$.
- ▶ Given relation schema R and a set of schemata $R_1, R_2, \dots, R_n$:
- ▶ we want to formalize the statement:
- ▶ R can be reconstructed just by natural join from $R_1, R_2, \dots, R_n$. iff for all legal $r(R)$:
$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) \bowtie \dots \bowtie \Pi_{R_n}(r)$$
- ▶ Different wording: R can be reconstructed by joining multiple projections of itself.
- ▶ We call this a Join Dependency.
- ▶ Syntax 1 : **JD($R, [R_1, R_2, \dots, R_n]$)**
- ▶ Syntax 2: $\ast(R_1, R_2, \dots, R_n)$ on schema R

Restate Aim

- ▶ If a relation schema R can be reconstructed just by natural join from a set of schemata $R_1, R_2, \dots, R_n$ with all of them subsets of R :
- ▶ then we observe a Join Dependency: $JD(R, [R_1, R_2, \dots, R_n])$
- ▶ then we prefer decomposition of the table, a database with relations based on $R_1, R_2, \dots, R_n$
- ▶ Note, this at first leads to a separate table for every attribute dependent on a key.
 - ▶ This is akin to a column store database.
 - ▶ This is a valid and sometimes preferred design. 有效的
 - ▶ However, tables with the same key can then be merged (joined) on demand.

Decomposition (Column Store Style)

- ▶ Decomposition of a schema R into schemas $R_1, R_2, \dots, R_n$.
- ▶ Consider the relation **Course(course_id, title, dept_name, credits)**.

- ▶ Tuples:

course_id	title	dept_name	credits
C1	Database Systems	CS	4
C2	Algorithms	CS	3
C3	Linear Algebra	Math	3

- ▶ The relation can be reconstructed by joining:
 - ▶ **Course1(course_id, title)**
 - ▶ **Course2(course_id, dept_name)**
 - ▶ **Course3(course_id, credits)**
- ▶ on **course_id**.

Conclusion

- ▶ Joins are powerful tools in database management.
- ▶ relations sometimes can be decomposed into smaller relations and reconstructed just by natural join of all these relations.
- ▶ Embrace joins to ensure data integrity and efficiency.
- ▶ In the model assumed in Relational Database Design, a relation can be constrained by business considerations.
- ▶ This constraint usually cannot be enforced in the database:
- ▶ This situation is considered non-optimal.
- ▶ We can infer heuristically join dependencies from database instances and the patterns we can see.
- ▶ If we observe a Join Dependency: $JD(R, [R_1, R_2, \dots, R_n])$
- ▶ then we generally prefer decomposition of the table, a database with relations based on $R_1, R_2, \dots, R_n$